

Distance Weighting Algorithm for TuneTrace

April 11, 2025

1 Introduction

TuneTrace uses a distance weighting algorithm to compare melodies based on their pitch direction, note values, and timing. To ensure fair comparisons, Dynamic Time Warping (DTW) distances are first normalized by dividing them by the length of the user’s melody. This step ensures that comparisons are scale-invariant before applying the weighting function.

1.1 Definition of Direction

The *direction* of a melody refers to the relative movement between consecutive notes. Given a sequence of notes $N_1, N_2, \dots, N_m$, the direction sequence d is defined as:

$$d_i = \begin{cases} 1, & \text{if } N_{i+1} > N_i \quad (\text{ascending}) \\ 0, & \text{if } N_{i+1} = N_i \quad (\text{no change}) \\ -1, & \text{if } N_{i+1} < N_i \quad (\text{descending}) \end{cases} \quad (1)$$

Here, d_i represents the movement between the i th and $(i + 1)$ th notes. Comparing these direction sequences helps evaluate how similar two melodies are in terms of pitch contour.

1.2 Definition of Note Values

Note values represent the pitch of each note, converted from standard notation (e.g., C2, G#1, Db0) into integers using the MIDI numbering system. The transformation is defined by:

$$\text{MIDI note} = \text{index}(\text{pitch class}) + (\text{octave} + 1) \times 12 \quad (2)$$

where the pitch class follows the sequence:

$$[\text{C}, \text{C}\#, \text{D}, \text{D}\#, \text{E}, \text{F}, \text{F}\#, \text{G}, \text{G}\#, \text{A}, \text{A}\#, \text{B}] \quad (3)$$

1.3 Definition of Timings

Timings refer to the duration (in seconds) that each note or rest is held. This temporal data is essential for comparing rhythmic patterns between melodies.

1.4 Definition of Resolution

Melody resolution refers to the time interval used for sampling pitch values. It acts as a hop length for models like CREPE, torchcrepe, and librosa to detect pitch at regular time steps. Higher resolution improves pitch detection accuracy but increases computational cost.

1.5 Breaking Down the Melodies of the Song

TuneTrace begins by isolating melodic elements using **Demucs**, a state-of-the-art deep learning model for source separation. Demucs decomposes the audio into stems such as vocals, drums, bass, and other instruments. This enables TuneTrace to focus specifically on melodic content by filtering out background noise and percussion.

After isolating the relevant stem, TuneTrace uses pitch detection models like **CREPE**, **torchcrepe**, and **librosa** to analyze the melody in detail. These models extract the pitch contour over time, identifying melodic *direction*, *note values*, and *timings* with high precision. The extracted data forms the basis of TuneTrace’s melody recognition system, allowing for accurate transcription, comparison, and database matching. When a new song is added to the database, it is processed into a MIDI file, with each MIDI track representing a stem analyzed by one of the above models.

1.6 Melody Comparison Using Constrained Dynamic Time Warping

To compare melodies efficiently, TuneTrace uses Dynamic Time Warping (DTW) with a constrained warping window set to 10% of the sequence length. The size of this sliding window is adjusted based on both the length of the user input and the *resolution* of each melody. For example, if the user input has a time step of 0.01 seconds and the database song uses a 0.05-second time step, the sliding window will be 5 times smaller to reflect the user’s higher resolution.

This approach allows for flexible alignment of sequences with slight differences in resolution, tempo, or articulation, while maintaining computational efficiency. The 10% constraint, combined with highly optimized parallel iterators written in **Rust**, significantly reduces the search space and avoids incorrect alignments. As a result, only structurally similar melodies are matched. This technique supports robust similarity scoring across thousands of candidates, even in the presence of expressive or rhythmic variation.

The computed DTW distances are referred to as d (direction), n (note values), and t (timings).

2 Distance Normalization and Weighting Function

The DTW distances for direction (d), note values (n), and timing (t) are normalized by dividing them by the user melody's length (L) to get deviation percentages:

$$d' = \frac{d}{L}, \quad n' = \frac{n}{L}, \quad t' = \frac{t}{L} \quad (4)$$

Then, each normalized distance is passed through the weighting function:

$$D = \text{weighting_function}(d'), \quad N = \text{weighting_function}(n'), \quad T = \text{weighting_function}(t') \quad (5)$$

2.1 Constraints

The weighting function $\text{weighting_function}(x)$ is designed to meet the following conditions:

1. $\text{weighting_function}(0) = 0$
2. $\text{weighting_function}(x_0) = y_0$
3. For $x > x_0$: $\text{weighting_function}(x + n) \approx \text{weighting_function}(x) \cdot b^{(n/s)}$ for all $n \geq 0$
4. For $x < x_0$: $\text{weighting_function}(x) \approx 0$

2.2 Factorization

To satisfy these constraints, $\text{weighting_function}(x)$ is constructed as the product of several components:

$$\text{Main Factor: } Mf(x) = b^{(x-x_0)/s} \quad (\text{ensures exponential growth})$$

$$\text{First Correcting Factor: } Fcf(x) = \frac{1}{1 + a^{(x_0-x)/s}} \quad (\text{limits growth for small } x)$$

$$\text{Second Correcting Factor: } Scf(x) = \frac{2}{1 + e^{-2x}} - 1 \quad (\text{ensures } \text{weighting_function}(0) = 0)$$

$$\text{Third Correcting Factor: } Tcf(x) = \frac{2y_0(1 + e^{-2x_0})}{1 - e^{-2x_0}} \quad (\text{ensures } \text{weighting_function}(x_0) = y_0)$$

The final function becomes:

$$\text{weighting_function}(x) = Mf(x) \cdot Fcf(x) \cdot Scf(x) \cdot Tcf(x) \quad (6)$$

2.3 Calculating Weighted Distances

By applying the weighting function to d' , n' , and t' , we compute the final weighted distances:

$$D = \text{weighting_function}(d'), \quad N = \text{weighting_function}(n'), \quad T = \text{weighting_function}(t') \quad (7)$$

These are used to calculate the total similarity score.

3 Combining Weighted Distances

The function `aggregate_weighted_distance( $D, N, T$ )` combines the weighted components into a single distance metric.

3.1 Definition

Using predefined weight coefficients W_d , W_n , and W_t , the aggregate distance is calculated as:

$$\text{aggregate_weighted_distance}(D, N, T) = \frac{DW_d + NW_n + TW_t}{W_d + W_n + W_t} \quad (8)$$

4 Similarity Function

The similarity between two melodies is defined as:

$$\text{sim}(\text{aggregate_weighted_distance}(D, N, T)) = \frac{100\%}{1 + \text{aggregate_weighted_distance}(D, N, T)} \quad (9)$$

This formula ensures that higher distances yield lower similarity percentages in a smooth, intuitive way.

5 Conclusion

This distance weighting algorithm provides a mathematically sound and interpretable similarity score for melody recognition. The design balances accuracy, efficiency, and flexibility. Future work may explore adaptive weighting based on real-world performance and user testing.